

**Walchand College of Engineering, Sangli**  
Department of Computer Science & Engineering  
**Computer Algorithm Laboratory**  
**Assignment No. 5 — Greedy Method**

**Name : Atharv Vinayak Kulkarni**

**PRN : 245109074**

**Class : TY B.Tech CSE**

## Question 1: Network Signal Propagation – Shortest Delay Path (Dijkstra)

### Problem Statement:

A telecom company models its relay network as a directed graph where each edge weight is the propagation delay (ms). Implement Dijkstra's algorithm to compute the minimum delay from source S to every other node.

### Approach:

approach\_1: relax edges repeatedly using Bellman-Ford style scanning. tc:  $O(V \cdot E)$  sc:  $O(V)$

approach\_2 (optimal): use a min-priority queue; always expand the unvisited node with the smallest tentative distance and relax its outgoing edges (Dijkstra). tc:  $O((V+E) \log V)$  sc:  $O(V+E)$

### Time Complexity:

$O((V+E) \log V)$  using a binary heap priority queue

### Space Complexity:

$O(V + E)$

### Code:

```
#include<iostream>
#include<vector>
#include<queue>
#include<climits>
#include<string>
using namespace std;
void dijkstra(vector<string>name, vector<vector<pair<int,int>>>adj, int src, int target){
    int n=name.size();
    vector<int>dist(n, INT_MAX);
    vector<int>parent(n, -1);
    dist[src]=0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>>pq;
    pq.push({0,src});
    while(!pq.empty()){
        auto [d,u]=pq.top();
        pq.pop();
        if(d>dist[u]) continue;
        for(auto &[v,w]:adj[u]){
            if(dist[u]!=INT_MAX && dist[u]+w<dist[v]){
                dist[v]=dist[u]+w;
                parent[v]=u;
                pq.push({dist[v],v});
            }
        }
    }
}
cout<<"Shortest distance from "<<name[src]<<" to "<<name[target]<<": ";
if(dist[target]==INT_MAX){
```

```

        cout<<"unreachable\n";
        return;
    }
    cout<<dist[target]<<" ms\n";
    cout<<"Shortest path: ";
    vector<int>path;
    for(int v=target;v!=-1;v=parent[v]) path.push_back(v);
    for(int i=path.size()-1;i>=0;i--){
        cout<<name[path[i]];
        if(i>0) cout<<"->";
    }
    cout<<"\n";
}
int main(){
    //Test Case 1
    {
        vector<string>name={"S","A","B","C","D"};
        vector<vector<pair<int,int>>>adj(5);
        adj[0]={{1,4},{2,8}};
        adj[1]={{2,2},{3,5}};
        adj[2]={{3,1},{4,7}};
        adj[3]={{4,3}};
        cout<<"Test Case 1\n";
        dijkstra(name,adj,0,4);
    }
    //Test Case 2
    {
        vector<string>name={"S","A","B","C"};
        vector<vector<pair<int,int>>>adj(4);
        adj[0]={{1,10},{2,3}};
        adj[2]={{1,2},{3,8}};
        adj[1]={{3,1}};
        cout<<"\nTest Case 2\n";
        dijkstra(name,adj,0,3);
    }
    //Test Case 3
    {
        vector<string>name={"S","A","B","C","D"};
        vector<vector<pair<int,int>>>adj(5);
        adj[0]={{1,5}};
        adj[1]={{2,4}};
        adj[2]={{3,2}};
        adj[3]={{4,1}};
        cout<<"\nTest Case 3\n";
        dijkstra(name,adj,0,4);
    }
    //Test Case 4
    {
        vector<string>name={"S","A","B","C","D"};
        vector<vector<pair<int,int>>>adj(5);
        adj[0]={{1,2},{2,1}};
        adj[1]={{3,2}};
        adj[2]={{3,6}};
        adj[3]={{4,3}};
        cout<<"\nTest Case 4\n";
        dijkstra(name,adj,0,4);
    }
    //Test Case 5-unreachable destination
    {
        vector<string>name={"S","A","B","C"};
        vector<vector<pair<int,int>>>adj(4);
        adj[0]={{1,4}};
        adj[1]={{2,5}};
        cout<<"\nTest Case 5\n";
        dijkstra(name,adj,0,3);
    }
    return 0;
}

```

### Theory Questions:

- 1) Shortest path results: the updated program runs 5 independent Dijkstra test cases, including an unreachable-destination case, and prints the shortest distance and path for each case.
- 2) Negative edge weight: No. Dijkstra greedily finalizes a node's distance the moment it is popped from the priority queue, assuming no later relaxation can ever improve it. A negative edge could still shorten the path to an already-finalized node, so the algorithm can return an incorrect (too large) distance in that case. Bellman-Ford should be used instead when negative weights are possible.
- 3) DAG shortest path via topological sort: Topologically sort the vertices, then process them in that order, relaxing all outgoing edges of each vertex exactly once. Because every edge goes from an earlier vertex to a later one in the topological order, each vertex's shortest distance is finalized before it is used to relax others, so a priority queue is not needed. Time complexity is  $O(V + E)$ , which is faster than Dijkstra's  $O((V+E) \log V)$ , since no heap operations are required.
- 4) Adding a faster link between two already-connected nodes: The full shortest path tree does NOT need to be recomputed from scratch. Only the distances of the endpoint of the new edge and any nodes reachable from it need to be re-checked and possibly updated (a localized re-relaxation, e.g. re-run Dijkstra/relaxation starting from the affected node with the existing distances as the starting point). This is much cheaper than recomputing everything, though in the worst case the update can still propagate to many downstream nodes.

### Output:

```
Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment
5 at 10:07:06 PM
mkdir -p build && g++ Q1_Dijkstra.cpp -o build/Q1_Dijkstra && ./build/Q1_Dijkstra
Test Case 1
Shortest distance from S to D: 10 ms
Shortest path: S -> A -> B -> C -> D

Test Case 2
Shortest distance from S to C: 6 ms
Shortest path: S -> B -> A -> C

Test Case 3
Shortest distance from S to D: 12 ms
Shortest path: S -> A -> B -> C -> D

Test Case 4
Shortest distance from S to D: 7 ms
Shortest path: S -> A -> C -> D

Test Case 5
Shortest distance from S to C: unreachable
```

## Question 2: Esports Arena – Match Scheduling (Activity Selection)

### Problem Statement:

A single main stage receives match requests, each with a start and end time. Select the maximum number of non-overlapping matches using the greedy Activity Selection approach. The updated code demonstrates 5 independent test cases.

### Approach:

approach\_1: try all subsets of matches and check for overlaps. tc:  $O(2^n)$

approach\_2 (greedy): sort activities by finish time; greedily pick an activity if its start time is not before the finish time of the last selected activity. The code applies this to 5 test cases. tc:  $O(n \log n)$  sc:  $O(n)$

### Time Complexity:

$O(n \log n)$  — dominated by the sort

### Space Complexity:

$O(n)$

### Code:

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
void activitySelection(vector<pair<int,int>>matches){
    sort(matches.begin(),matches.end(),[](const pair<int,int>&a,const pair<int,int>&b){
        return a.second<b.second;
    });
    vector<pair<int,int>>selected;
    int lastEnd=-1;
    for(auto &m:matches){
        if(m.first>=lastEnd){
            selected.push_back(m);
            lastEnd=m.second;
        }
    }
    cout<<"Total activities: "<<matches.size()<<endl;
    cout<<"Maximum non-overlapping activities: "<<selected.size()<<endl;
    cout<<"Selected activities: ";
    for(auto &m:selected)
        cout<<"("<<m.first<<","<<m.second<<") ";
    cout<<endl;
}
int main(){
    //Test Case 1
    cout<<"Test Case 1\n";
    activitySelection({{1,3},{2,5},{4,7},{6,9},{8,10}});
    //Test Case 2
    cout<<"\nTest Case 2\n";
    activitySelection({{0,6},{1,2},{3,4},{5,7},{8,9},{5,9}});
    //Test Case 3
    cout<<"\nTest Case 3\n";
    activitySelection({{1,2},{2,3},{3,4},{4,5},{5,6}});
    //Test Case 4
    cout<<"\nTest Case 4\n";
    activitySelection({{1,10},{2,3},{3,4},{5,7},{8,9},{10,12}});
```

```
//Test Case 5
cout<<"\nTest Case 5\n";
activitySelection({{1,4},{2,3},{3,5},{5,7},{6,8},{8,10},{9,11}});
return 0;
}
```

### Output:

```
*Linux@atharvk %main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_5
at 10:07:49 PM
↳ mkdir -p build && g++ Q2_ActivitySelection.cpp -o build/Q2_ActivitySelection && ./build/Q2_ActivitySelection

Test Case 2
Total activities: 6
Maximum non-overlapping activities: 4
Selected activities: (1,2) (3,4) (5,7) (8,9)

Test Case 3
Total activities: 5
Maximum non-overlapping activities: 5
Selected activities: (1,2) (2,3) (3,4) (4,5) (5,6)

Test Case 4
Total activities: 6
Maximum non-overlapping activities: 5
Selected activities: (2,3) (3,4) (5,7) (8,9) (10,12)

Test Case 5
Total activities: 7
Maximum non-overlapping activities: 4
Selected activities: (2,3) (3,5) (5,7) (8,10)
```

## Question 3: Corporate Office – Minimum Meeting Rooms Required

### Problem Statement:

Given meeting requests with start and end times, determine the minimum number of rooms required so that no two meetings in the same room overlap. The updated code demonstrates 5 independent test cases.

### Approach:

approach\_1: for every meeting, check all rooms currently in use to find a free one. tc:  $O(n^2)$

approach\_2 (greedy): separately sort all start times and all end times; sweep through with two pointers, incrementing the active-room counter on a start and decrementing on an end. The code applies this to 5 test cases. tc:  $O(n \log n)$  sc:  $O(n)$

### Time Complexity:

$O(n \log n)$

### Space Complexity:

$O(n)$

### Code:

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
void minMeetingRooms(vector<pair<int,int>>meetings){
    int n=meetings.size();
    vector<int>start(n),end_(n);
    for(int i=0;i<n;i++){
        start[i]=meetings[i].first;
        end_[i]=meetings[i].second;
    }
    sort(start.begin(),start.end());
    sort(end_.begin(),end_.end());
    int rooms=0,maxRooms=0;
    int i=0,j=0;
    while(i<n && j<n){
        if(start[i]<end_[j]){
            rooms++;
            i++;
            maxRooms=max(maxRooms,rooms);
        }else{
            rooms--;
            j++;
        }
    }
    cout<<"Total meetings: "<<n<<endl;
    cout<<"Minimum meeting rooms required: "<<maxRooms<<endl;
}
int main(){
    //Test Case 1
    cout<<"Test Case 1\n";
    minMeetingRooms({{900,1000},{930,1030},{1000,1100},{1030,1130}});
    //Test Case 2
    cout<<"\nTest Case 2\n";
    minMeetingRooms({{900,1000},{1000,1100},{1100,1200}});
    //Test Case 3
```

```

        cout<<"\nTest Case 3\n";
        minMeetingRooms({{900,1200},{930,1100},{1000,1130},{1030,1230}});
        //Test Case 4
        cout<<"\nTest Case 4\n";
        minMeetingRooms({{800,900},{850,950},{900,1000},{920,1030},{1000,1100}});
        //Test Case 5
        cout<<"\nTest Case 5\n";
        minMeetingRooms({{900,930},{930,1000},{1000,1030},{1030,1100},{1100,1130}});
        return 0;
    }

```

## Output:

```

Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_
5 at 10:08:20 PM
└─> mkdir -p build && g++ Q3_MinMeetingRooms.cpp -o build/Q3_MinMeetingRooms && ./build/Q3_MinMeetingRooms
Test Case 1
Total meetings: 4
Minimum meeting rooms required: 2

Test Case 2
Total meetings: 3
Minimum meeting rooms required: 1

Test Case 3
Total meetings: 4
Minimum meeting rooms required: 4

Test Case 4
Total meetings: 5
Minimum meeting rooms required: 3

Test Case 5
Total meetings: 5
Minimum meeting rooms required: 1

```

## Question 4: Shared Office Printer – Minimizing Total Waiting Time

### Problem Statement:

A printer has a queue of jobs with different durations. Decide the printing order that minimizes total (and average) waiting time. The updated code demonstrates 5 independent test cases.

### Approach:

approach\_1: process jobs in the order they were submitted (FCFS) and compute waiting times. Not optimal in general.

approach\_2 (greedy): Shortest Job First — sort jobs by duration ascending and print in that order. This minimizes total/average waiting time in a single-server queue. The code applies this to 5 test cases. tc:  $O(n \log n)$  sc:  $O(n)$

### Time Complexity:

$O(n \log n)$

### Space Complexity:

$O(n)$

### Code:

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
void minWaitingTime(vector<int>jobs){
    int n=jobs.size();
    sort(jobs.begin(),jobs.end());
    vector<int>waitTime(n,0);
    long long totalWait=0;
    for(int i=1;i<n;i++){
        waitTime[i]=waitTime[i-1]+jobs[i-1];
        totalWait+=waitTime[i];
    }
    cout<<"Total jobs: "<<n<<endl;
    cout<<"Order of printing: ";
    for(int d:jobs) cout<<d<<" ";
    cout<<endl;
    cout<<"Waiting times: ";
    for(int i=0;i<n;i++)
        cout<<waitTime[i]<<" ";
    cout<<endl;
    cout<<"Total waiting time="<<totalWait<<" sec"<<endl;
    cout<<"Average waiting time="<<(double)totalWait/n<<" sec"<<endl;
}
int main(){
    //Test Case 1
    cout<<"Test Case 1\n";
    minWaitingTime({45,120,30,80,200});
    //Test Case 2
    cout<<"\nTest Case 2\n";
    minWaitingTime({10,20,30,40});
    //Test Case 3
    cout<<"\nTest Case 3\n";
    minWaitingTime({100,1,50,25,75});
    //Test Case 4
    cout<<"\nTest Case 4\n";
```



```
minWaitingTime({300,200,100,50,10,5});  
//Test Case 5  
cout<<"\nTest Case 5\n";  
minWaitingTime({60,60,20,40,80});  
return 0;  
}
```

### Output:

```
Linux@atharvk % main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_5 at 10:08:55 PM  
└─> mkdir -p build && g++ Q4_MinWaitingTime.cpp -o build/Q4_MinWaitingTime && ./build/Q4_MinWaitingTime  
Test Case 1  
Total jobs: 5  
Order of printing: 30 45 80 120 200  
Waiting times: 0 30 75 155 275  
Total waiting time = 535 sec  
Average waiting time = 107 sec  
  
Test Case 2  
Total jobs: 4  
Order of printing: 10 20 30 40  
Waiting times: 0 10 30 60  
Total waiting time = 100 sec  
Average waiting time = 25 sec  
  
Test Case 3  
Total jobs: 5  
Order of printing: 1 25 50 75 100  
Waiting times: 0 1 26 76 151  
Total waiting time = 254 sec  
Average waiting time = 50.8 sec  
  
Test Case 4  
Total jobs: 6  
Order of printing: 5 10 50 100 200 300  
Waiting times: 0 5 15 65 165 365  
Total waiting time = 615 sec  
Average waiting time = 102.5 sec  
  
Test Case 5  
Total jobs: 5  
Order of printing: 20 40 60 60 80  
Waiting times: 0 20 60 120 180  
Total waiting time = 380 sec  
Average waiting time = 76 sec
```

## Question 5: Marketing Campaign – Minimum Channel Coverage (Set Cover)

### Problem Statement:

Demographic segments must all be reached using as few advertising channels as possible, where each channel covers a fixed subset of segments. Use the greedy Set Cover approximation algorithm. The updated code demonstrates 5 independent test cases.

### Approach:

approach\_1 (exact): try all subsets of channels and check coverage. tc:  $O(2^m)$ , NP-hard in general.

approach\_2 (greedy approximation): repeatedly select the channel that covers the largest number of currently-uncovered segments until all segments are covered or no further progress is possible. The code applies this to 5 test cases, including an impossible-coverage case. tc:  $O(m^2 * s)$  in the straightforward implementation.

### Time Complexity:

$O(m^2 * s)$  for the greedy approximation ( $m$  = number of channels)

### Space Complexity:

$O(m * s)$

### Code:

```
#include<iostream>
#include<vector>
#include<set>
using namespace std;
void setCover(int totalSegments,vector<set<int>>channels){
    set<int>universe;
    for(int i=1;i<=totalSegments;i++)
        universe.insert(i);
    set<int>covered;
    vector<int>chosenChannels;
    while(covered.size()<universe.size()){
        int bestChannel=-1;
        int bestGain=0;
        set<int>bestNew;
        for(int i=0;i<(int)channels.size();i++){
            set<int>newlyCovered;
            for(int seg:channels[i])
                if(covered.find(seg)==covered.end())
                    newlyCovered.insert(seg);
            if((int)newlyCovered.size()>bestGain){
                bestGain=newlyCovered.size();
                bestChannel=i;
                bestNew=newlyCovered;
            }
        }
        if(bestChannel==-1) break;
        chosenChannels.push_back(bestChannel+1);
        for(int seg:bestNew)
            covered.insert(seg);
    }
    cout<<"Universe size: "<<totalSegments<<endl;
    cout<<"Channels selected: "<<chosenChannels.size()<<endl;
    cout<<"Selected channels: ";
    for(int c:chosenChannels)
        cout<<"Channel"<<c<<" ";
}
```

```
cout<<endl;
if(covered.size()<universe.size())
cout<<"Warning: some segments could not be covered"<<endl;
else
cout<<"All segments covered"<<endl;
}
int main(){
    //Test Case 1
    cout<<"Test Case 1\n";
    setCover(6,{
        {1,2,3},{2,4},{3,5},{4,5,6},{1,6}
    });
    //Test Case 2
    cout<<"\nTest Case 2\n";
    setCover(8,{
        {1,2,3},{3,4,5},{5,6},{6,7,8},{1,8}
    });
    //Test Case 3
    cout<<"\nTest Case 3\n";
    setCover(5,{
        {1,2},{2,3},{3,4},{4,5},{1,5}
    });
    //Test Case 4
    cout<<"\nTest Case 4\n";
    setCover(10,{
        {1,2,3,4},{4,5,6},{6,7,8},{8,9,10},{2,5,9}
    });
    //Test Case 5-impossible to cover all segments
    cout<<"\nTest Case 5\n";
    setCover(7,{
        {1,2},{2,3},{4,5},{5,6}
    });
    return 0;
}
```

## Output:

```
Linux@atharvk main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_5 at 10:10:04 PM
↳ mkdir -p build && g++ Q5_SetCover.cpp -o build/Q5_SetCover && ./build/Q5_SetCover
Test Case 1
Universe size: 6
Channels selected: 2
Selected channels: Channel1 Channel4
All segments covered

Test Case 2
Universe size: 8
Channels selected: 3
Selected channels: Channel1 Channel4 Channel2
All segments covered

Test Case 3
Universe size: 5
Channels selected: 3
Selected channels: Channel1 Channel3 Channel4
All segments covered

Test Case 4
Universe size: 10
Channels selected: 4
Selected channels: Channel1 Channel3 Channel4 Channel2
All segments covered

Test Case 5
Universe size: 7
Channels selected: 4
Selected channels: Channel1 Channel3 Channel2 Channel4
```

## Question 6: ATM Cash Dispenser – Minimum Notes Problem

### Problem Statement:

Dispense a requested withdrawal amount using the minimum number of currency notes chosen from the available denominations. The updated code demonstrates 5 test cases, including a counterexample where greedy is not optimal and a case where the exact amount cannot be formed.

### Approach:

approach\_1 (exact / DP): classic unbounded coin-change DP that tries every denomination for every amount. tc:  $O(\text{amount} * \text{denominations})$

approach\_2 (greedy): sort denominations descending; repeatedly take as many notes of the largest usable denomination as possible. The code applies this to 5 test cases. It is optimal for canonical currency systems but not for arbitrary denomination sets. tc:  $O(d \log d)$  sc:  $O(d)$  including the denomination copy and output storage.

### Time Complexity:

$O(d \log d)$  for the greedy approach ( $d$  = number of denominations)

### Space Complexity:

$O(1)$  extra (excluding output storage)

### Code:

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
void minNotes(vector<int>denom,int amount){
    sort(denom.rbegin(),denom.rend());
    vector<pair<int,int>>used;
    int totalNotes=0;
    int remaining=amount;
    for(int d:denom){
        if(remaining<=0) break;
        int count=remaining/d;
        if(count>0){
            used.push_back({d,count});
            totalNotes+=count;
            remaining-=d*count;
        }
    }
    cout<<"Amount: "<<amount<<endl;
    if(remaining!=0){
        cout<<"Cannot form the exact amount with the given denominations.\n";
        return;
    }
    cout<<"Notes used: ";
    for(auto &p:used)
        cout<<p.first<<"x"<<p.second<<" ";
    cout<<endl;
    cout<<"Total notes used: "<<totalNotes<<endl;
}
int main(){
    //Test Case 1-standard denominations
    cout<<"Test Case 1\n";
    minNotes({1,2,5,10,20,50,100,200,500},1287);
}
```

```

//Test Case 2
cout<<"\nTest Case 2\n";
minNotes({1,5,10,20,50,100},275);
//Test Case 3
cout<<"\nTest Case 3\n";
minNotes({1,2,5,10,20},47);
//Test Case 4-greedy is not optimal for arbitrary denominations
cout<<"\nTest Case 4\n";
minNotes({1,3,4},6);
cout<<"Note: Greedy gives 4+1+1=3 notes, while optimal is 3+3=2 notes.\n";
//Test Case 5-exact amount cannot be formed
cout<<"\nTest Case 5\n";
minNotes({5,10,20},13);
return 0;
}

```

### Output:

```

Linux@atharvk ~$ main in ~/Documents/WCE_Lab_Material/TY_Computer_Algorithm_Lab/245109074_CAL_Assignment_
5 at 10:10:48 PM
└─> mkdir -p build && g++ Q6_MinNotes.cpp -o build/Q6_MinNotes && ./build/Q6_MinNotes
Test Case 1
Amount: 1287
Notes used: 500x2 200x1 50x1 20x1 10x1 5x1 2x1
Total notes used: 8

Test Case 2
Amount: 275
Notes used: 100x2 50x1 20x1 5x1
Total notes used: 5

Test Case 3
Amount: 47
Notes used: 20x2 5x1 2x1
Total notes used: 4

Test Case 4
Amount: 6
Notes used: 4x1 1x2
Total notes used: 3
Note: Greedy gives 4+1+1 = 3 notes, while optimal is 3+3 = 2 notes.

Test Case 5
Amount: 13
Cannot form the exact amount with the given denominations.

```